

AY 2025/2026



POLITECNICO
MILANO 1863

Internet Of Things: Homework Exercise 2

Alberto Biffi - 10677341

Professor
Matteo CESANA Antonio BOIANO Fabio PALMESE

Version 1.0
May 28, 2026

Contents

1	Assumptions	2
2	Question 1	2

1 Assumptions

A nursing home uses an IoT system to help caregivers localize dental prostheses of Alzheimer patients.

Each denture contains a small BLE beacon that periodically transmits advertising packets. BLE gateways installed in the nursing home receive the packets and estimate the approximate position of the denture based on RSSI measurements.

The current firmware is the following:

```
1 loop forever:
2     send_BLE_beacon()
3     sleep (10 seconds)
```

The objective is to improve the firmware and the sensing architecture in order to increase battery lifetime while preserving the localization service.

2 Question 1

Design an improved IoT solution.

Your answer should include:

1. The logic used to for BLE beaconing;
2. Any additional sensors added to the system;
3. Pseudocode of the improved firmware;
4. One or more real commercial components that could be used to implement the proposed sensing logic;
5. An estimate of the expected energy saving with respect to the original periodic beaconing solution.

In the energy-saving estimate, clearly state all assumptions including the average time per day in which the denture is expected to require localization.

The original system is highly inefficient because it sends a BLE beacon every 10 seconds regardless of the denture's actual state. We can use a simple logic with 2 states to reduce the consumption. If the denture is in motion, it means that it's being worn by the patient, otherwise, if it is stationary, then it means that it is removed and placed somewhere.

To do so we need to add a MEMS accelerometer that will detect the state of the denture. This is an example of pseudocode that must be implemented:

```
1 function setup_system():
2     //Initialization of BLE and Sensor
3     //Interrupt for motion/stationary
4 main():
5     loop forever:
6         send_BLE_beacon()
7         if denture_state == IN_MOTION:
8             sleep(5 minutes)
9         else:
10            sleep(10 seconds)
```

For the microcontroller we could use the Dialog DA14531, which is one of the smallest and most efficient BLE chips. (<https://www.renesas.com/en/products/da14531>) The industry standard MEMS accelerator is the ADXL345, but for Ultra-Low Power, the best choice is the ADXL362 with only 270 nA motion-activated wake-up mode. (<https://www.analog.com/en/products/adxl362.html>) For the battery, we could use a small rechargeable lithium battery with an added coil for wireless recharge, like the Varta CoinPower CP 0854.

Before our implementation the system was supposed to send a beacon every 10 seconds, which means 6 beacons per minute, 360 beacons per hour, and 8640 beacons per day. Now we can make some assumptions: for the patient, we suppose that the denture is worn for 16 hours per day, which is the patient's average awake time, and is therefore in the In-Motion state for this amount of time. The rest of the time the patient is supposed as sleep, and so the denture is removed, and is therefore in the Stationary state. For the energy consumption, we can assume that it is almost entirely dominated by the BLE radio transmissions. Therefore, we can not consider the ADXL362 energy consumption.

During the In-Motion state, the system transmits 1 beacon every 5 minutes, 12 beacons per hour, and 192 beacons in the 16 hours of the state. During the Stationary state, the system transmits 1 beacon every 10 seconds, which results in a total of 2880 beacons during the 8 hours of sleep.

This results in a reduction of the transmissions of the $\frac{8640}{192+2880} \cdot 100 = 64.4\%$. This will also reduce the energy consumption, thereby proportionally extending the battery lifetime.

With a battery like the Varta CoinPower CP0854, we could have a battery with a capacity of around 25 mAh which could easily last for days or weeks before needing recharging. For this reason, I decided to keep the system logic streamlined by avoiding additional components, such as temperature or humidity sensors. Relying solely on the accelerometer makes the design highly viable and efficient for real-world deployment.